

```
1.  #include <stdio.h>
2.  #include <stdlib.h>
3.  #include <string.h>
4.
5.  int main()
6.  {
7.      printf("Runtime Error Practice Program\n");
8.
9.      int a = 100;
10.     int b = 0;
11.
12.     printf("Division Result = %d\n", a / b);
        // RUNTIME ERROR: Division by zero.
        // CORRECTION:
        // if(b != 0)
        //     printf("Division Result = %d\n", a / b);
        // else
        //     printf("Cannot divide by zero\n");
13.
14.     int *ptr1 = NULL;
15.
16.     *ptr1 = 50;
        // RUNTIME ERROR: Dereferencing a NULL pointer.
        // CORRECTION:
        // int value = 50;
        // ptr1 = &value;
17.
18.     int arr[5];
19.
20.     arr[10] = 100;
        // RUNTIME ERROR: Array index out of bounds.
        // CORRECTION: arr[4] = 100;
21.
22.     int *ptr2;
23.
24.     *ptr2 = 200;
        // RUNTIME ERROR: Dereferencing an uninitialized pointer.
        // CORRECTION:
        // int value2;
        // ptr2 = &value2;
25.
```

```

26. char name[5];
27.
28. strcpy(name, "Programming");
    // RUNTIME ERROR: Buffer overflow.
    // CORRECTION:
    // char name[12] = "Programming";
29.
30. printf("%s\n", name);
31.
32. int *ptr3;
33.
34. printf("%d\n", *ptr3);
    // RUNTIME ERROR: Dereferencing an uninitialized pointer.
    // CORRECTION:
    // int value3 = 0;
    // ptr3 = &value3;
35.
36. FILE *fp;
37.
38. fp = fopen("unknown.txt", "r");
39.
40. fscanf(fp, "%d", &a);
    // RUNTIME ERROR: fp may be NULL.
    // CORRECTION:
    // if(fp != NULL)
    //     fscanf(fp, "%d", &a);
41.
42. fclose(fp);
    // RUNTIME ERROR: Closing a NULL file pointer.
    // CORRECTION:
    // if(fp != NULL)
    //     fclose(fp);
43.
44. char *ptr4 = malloc(5);
45.
46. strcpy(ptr4, "Computer Science");
    // RUNTIME ERROR: Buffer overflow.
    // CORRECTION:
    // ptr4 = malloc(strlen("Computer Science") + 1);
47.
48. printf("%s\n", ptr4);

```

```
49.
50.  free(ptr4);
51.
52.  printf("%c\n", ptr4[0]);
    // RUNTIME ERROR: Use-after-free.
    // CORRECTION:
    // ptr4 = NULL;
53.
54.  int *ptr5 = malloc(sizeof(int));
55.
56.  free(ptr5);
57.
58.  free(ptr5);
    // RUNTIME ERROR: Double free.
    // CORRECTION:
    // free(ptr5);
    // ptr5 = NULL;
59.
60.  int x = 5;
61.
62.  int y = 0;
63.
64.  int result = x % y;
    // RUNTIME ERROR: Modulo by zero.
    // CORRECTION:
    // if(y != 0)
    //     result = x % y;
65.
66.  printf("%d\n", result);
67.
68.  char str1[10] = "Hello";
69.
70.  strcat(str1, "WorldProgramming");
    // RUNTIME ERROR: Buffer overflow.
    // CORRECTION:
    // char str1[25] = "Hello";
    // strcat(str1, "World");
71.
72.  printf("%s\n", str1);
73.
74.  int *numbers;
```

```
75.
76.  numbers = malloc(3 * sizeof(int));
77.
78.  for(int i = 0; i < 10; i++)
79.  {
80.  numbers[i] = i;
    // RUNTIME ERROR: Writing beyond allocated memory.
    // CORRECTION: for(int i = 0; i < 3; i++)
81.  }
82.
83.  free(numbers);
84.
85.  int matrix[3][3];
86.
87.  matrix[5][5] = 100;
    // RUNTIME ERROR: Matrix index out of bounds.
    // CORRECTION: matrix[2][2] = 100;
88.
89.  int *ptr6 = NULL;
90.
91.  printf("%d\n", ptr6[0]);
    // RUNTIME ERROR: Dereferencing a NULL pointer.
    // CORRECTION:
    // int value6 = 10;
    // ptr6 = &value6;
92.
93.  char password[8];
94.
95.  gets(password);
    // RUNTIME ERROR: gets() is unsafe and removed from modern C.
    // CORRECTION:
    // fgets(password, sizeof(password), stdin);
96.
97.  printf("%s\n", password);
98.
99.  int *ptr7 = malloc(sizeof(int));
100.
101.  *ptr7 = 500;
102.
103.  free(ptr7);
104.
```

```

105.  *ptr7 = 1000;
      // RUNTIME ERROR: Use-after-free.
      // CORRECTION:
      // ptr7 = NULL;
106.
107.  int age;
108.
109.  printf("Age = %d\n", age);
      // RUNTIME ERROR: Using an uninitialized variable.
      // CORRECTION: int age = 0;
110.
111.  int index = -1;
112.
113.  arr[index] = 50;
      // RUNTIME ERROR: Negative array index.
      // CORRECTION:
      // if(index >= 0 && index < 5)
114.
115.  int *ptr8;
116.
117.  if(*ptr8 > 0)
      // RUNTIME ERROR: Dereferencing an uninitialized pointer.
      // CORRECTION:
      // int value8 = 5;
      // ptr8 = &value8;
118.  {
119.    printf("Positive\n");
120.  }
121.
122.  char *text = NULL;
123.
124.  printf("%s\n", text);
      // RUNTIME ERROR: Passing NULL to printf("%s").
      // CORRECTION:
      // if(text != NULL)
      //   printf("%s\n", text);
125.
126.  int size = 1000000000;
127.
128.  int *hugeArray;
129.

```

```
130. hugeArray = malloc(size * sizeof(int));
131.
132. hugeArray[0] = 1;
    // RUNTIME ERROR: malloc() may return NULL.
    // CORRECTION:
    // if(hugeArray != NULL)
    //   hugeArray[0] = 1;
133.
134. free(hugeArray);
135.
136. int *ptr9 = NULL;
137.
138. while(1)
139. {
140.   ptr9++;
    // RUNTIME ERROR: Infinite loop and invalid pointer arithmetic.
    // CORRECTION: Remove the loop or use a valid pointer.
141. }
142.
143. printf("End of Program\n");
144.
145. return 0;
146. }
```